_> BASH SCRIPTING BASICS

STATION: MAN-PAGE-CORE // EXECUTABLE REFERENCE GUIDE

---

1. The Absolute Basics

Shebang (#!)

Every Bash script must start with a shebang on the very first line. This tells the operating system which interpreter to use to execute the file.

```
#!/bin/bash
```

Execution Permissions

By default, new files aren't executable. You must grant permission using `chmod` and run them using `./` (which specifies the current directory):

```
$ chmod +x script.sh
$ ./script.sh
```

Comments

Any text following a `#` is ignored by the shell.

```
# This is a single-line comment
```

2. Variables and Input/Output

Declaring & Using Variables

CRITICAL RULE: No spaces around the `=` sign during assignment. Use the `$` sign to reference or read a variable.

```
name="Stewart"   # Correct
name = "Stewart" # WRONG (Shell thinks 'name' is a command)

echo "Hello, $name"
```

User Input

Use the `read` command to capture interactive user input.

```
echo "Enter your target IP:"
read target_ip
```

3. Conditional Statements (if/else)

Bash uses the structural syntax: `if [ condition ]; then ... fi`.

CRITICAL RULE: You must leave spaces inside the square brackets `[ ]`, or the script will crash.

```
if [ "$name" = "Stewart" ]; then
    echo "Access granted."
elif [ "$name" = "Alex" ]; then
    echo "Limited access granted."
else
    echo "Access denied."
fi
```

Common Comparison Operators

Test Type	Operator	Meaning	Example
String	= / ==	Equals	`[ "$a" = "$b" ]`
	!=	Not equals	`[ "$a" != "$b" ]`
	-z	String is empty	`[ -z "$a" ]`
Numeric	-eq	Equal to	`[ "$x" -eq 10 ]`
	-ne	Not equal to	`[ "$x" -ne 10 ]`
	-lt	Less than	`[ "$x" -lt 10 ]`
	-gt	Greater than	`[ "$x" -gt 10 ]`
File	-e	File exists	`[ -e "file.txt" ]`
	-d	Is a directory	`[ -d "/var/log" ]`

4. Loops

Loops let you repeat code blocks cleanly. They always open with **do** and close with **done**.

For Loop (Over a Range or List)

```
# Iterating through a fixed range (1 to 5)
for i in {1..5}; do
    echo "Count: $i"
done

# Iterating through a list of strings
for host in google.com pastebin.com thm.com; do
    ping -c 1 "$host"
done
```

While Loop (Based on a Condition)

Runs as long as the specified condition remains true.

```
counter=1
while [ $counter -le 3 ]; do
    echo "Iteration $counter"
    counter=$((counter + 1)) # Arithmetic expansion syntax
done
```

5. Functions

Functions allow you to group code chunks into reusable blocks.

```
# Define the function
log_message() {
    echo "[LOG] $(date): $1" # $1 represents the first argument passed
}

# Call the function
log_message "Script started successfully."
log_message "Database connection established."
```

6. Special Built-in Variables

Bash automatically populates these variables during script execution:

- **$0** – The name of the script itself.
- **$1, $2, ... $9** – The positional arguments passed to the script (e.g., `./script.sh arg1 arg2`).
- **$#** – The total number of arguments passed to the script.

- **$?** – The exit status of the last executed command (0 means success, anything else means an error occurred).
- **$$** – The Process ID (PID) of the current script.